Autonomous Software Agents Project Report

Daniele Buondonno
267888
daniele.buondonno@studenti.unitn.it

Sasha Petkovic
264689
sasha.petkovic@studenti.unitn.it

1 Introduction

In this report, we will give an overview of our project for the Autonomous Software Agents course. The project's goal is to create and develop autonomous agents that can play the game Deliveroo.js.
The game consists in picking up and deliverying parcels, which yield different reward values. The game's environment is competitive, as rival agents can be present. The project is divided into two parts.

- BDI Agent: An autonomous agent that uses the Belief-Desire-Intention architecture to observe and perform actions in an environment. This architecture allows the agent to maintain a set of beliefs about the environment obtained through sensing. Through the belief set, the architecture allows the agent to compute possible options (desires) along a utility score for them. The option with the best utility score is selected and performed, if possible.
  The architecture prescribes that a loop is present, where options continue to be generated given new observations, and where the current Intention can be revised.

- LLM Agent: An agent that acts as a partner to the BDI agent. It performs like the BDI agent through a setup composed of API calls and prompts. It must be able to understand the rules of the game and the environment around it.
  It must also be able to understand, evaluate and perform missions delivered to it through the game chat, which can yield positive or negative results.

We will first highlight the structure of the BDI agent, explaining its different components and operations. We will go through the data structures used, the strategies we decided to implement and our reasoning behind both. We will then explain the LLM's agent structure and operations. To conclude we will give some brief results and our thoughts for further improving our solutions.

2 Belief

In the Belief-Desire-Intention architecture, an agent's belief set represents what it believes to be true about the environment. The term belief is not used casually: what the set contains is not ground truth, but what the agent has deduced from present and past observations, which may no longer hold. As we will see, in our solution we try to keep the most up to date informations and forget what we confirm to be outdated.
We will now go over the different classes implementing the agent's Belief Set. Every class is instantiated per agent rather than shared at module level, which is what allows the BDI and the LLM agent to run in the same process.

2.1 Me and Agents

The Me class holds the agent's identity, its position and its score. For the agents' positions, we must consider when gating or considering fractional positions, as they indicate that an agent is attempting to move or is in the middle of one. For the agent itself, we gate fractional positions, avoiding inconsistent information about our own position and making wrong assumptions about a move through the environment.
The class Agents holds the latest observed state of every other agent, indexed by id. For this class we save fractional positions because they tell us that two tiles are occupied rather than one. In fact, a stationary agent occupies only one tile, while a moving agent occupies both its current tile and the tile it's attempting to move towards.

2.2 Parcels

This class represents what the agent believes about the parcels it carries or has observed, using three maps indexed by parcel id.
In **Visible** we store all the parcels the agents saw during the latest sensing and it is rebuilt every time the agent performs sensing. These observed parcels are the ones the agent is most confident about, as it can currently see them.
Known contains free parcels observed in the past, together with the time of their last observation. This timestamp is used to compute estimate parcel reward when decay is active.
To maintain this data structure as accurate as possible, whenever a known parcel's location is observed, and the parcel is not present, it is removed and forgotten by the agent. If the agent has a partner, this map also holds the information of the parcels reported by the partner. A report does not overwrite an already known parcel.
Carried holds the parcels the agent's currently carrying. It is updated through sensing and through confirmed pickup and putdown actions.

2.3 Crates

This class is specifically designed to represent the agent's beliefs about the crates in the map, if present. A **Known** map is also used here, it works the same way as for the parcels, only this time it stores crates.
We also made use of a **byPosition** map, indexed by a coordinate string. **Known** remains the main belief representation, while the positional index allows to avoid repeatedly scanning all known crates during planning.

2.4 Partner

This class stores the latest information received from the teammate and manages the data exchanged between the two agents. The shared state includes position, carried parcel ids, carried count and carried reward. The agents also exchange observed free parcels and announce the parcel they are currently pursuing.
Before committing to a pickup, an agent compares its distance to the parcel with the distance announced by the partner and yields if the partner is closer. Ties are resolved through a deterministic ordering of the agent ids. Their communication is overhead free as they only send new information to each other, avoiding duplicate information.

2.5 World

This class stores the map, the current configuration, the positions covered by the latest sensing event, and the known spawner and delivery tiles.
An important derived quantity is **decayPerMove**, which expresses reward decay in terms of movement cost.

$$decayPerMove = \begin{cases} \frac{movementDuration}{decayInterval} & \text{if } decayInterval > 0 \\ 0 & \text{otherwise} \end{cases}$$

This conversion allows pickup and delivery utilities to estimate reward loss directly from shortest path length.
Each spawner and delivery tile stores the time it was last observed and whether it belongs to a topology that supports a repeatable pickup-delivery cycle. We keep this information for maps with one way regions: a tile may be reachable but it can render the agent unable to return to a useful spawn or delivery area. These delivery tiles are prioritised during computations, while all reachable delivery tiles remain available as a fallback. Exploration is conservative and considers only spawners from which an operational delivery can still be reached.

2.6 Action Based Belief Revision

Beliefs are revised not only through sensing, but also through the confirmed outcome of actions. Leaving the belief revision only to observations can make the next planning cycle use a state that is outdated. Pickup and putdown outcomes revise the visible, known and carried parcel maps, while a successful crate push updates both crate indexes. These revisions are applied only after the action reports success.

2.7 Evidence Based Belief Revision

Our belief state preserves the latest information, it does not make use of histories or probabilistic values. Enemy agents are represented only by the latest sensing, no future trajectories are predicted.
Parcels and crates are treated differently, as their last known positions remain useful outside the current field of view. They are retained until the belief gets contradicted or, for parcels, the estimated reward becomes negative. This is an intentional tradeoff. A predictive model could produce an incorrect extrapolation that can suppress valid intentions.

3 Desires and Intention Loop

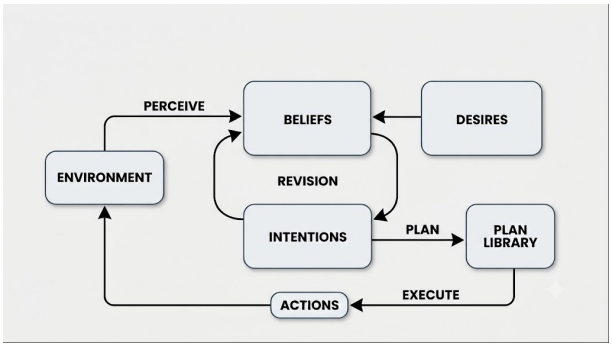

Figure 1: The BDI Loop.

During every BDI iteration, the agent generates desires that are currently achievable given its beliefs. Each desire contains a type, a target, an estimated utility and the identifiers required for planning.
The agent supports three autonomous desire types:

- **go_pick_up**: it defines the desire of picking up a parcel.
- **go_deliver**: it defines the desire of deliverying the currently carried parcels.
- **go_to_spawner**: this type of desire is used as an exploration option. It is designed to be generated only when the agent currently doesn't have the option of either deliverying or picking up parcels.

The LLM layer may also introduce temporary external objectives. They enter the same loop as standard desires, while their interpretation and creation are described in the LLM section.

3.1 Desires Generation

Utility estimation uses shortest path distances through a BFS search. A BFS from the current position provides the distance to every reachable pickup, delivery and exploration candidate. For each possible pickup tile, another search evaluates the following routes towards available delivery tiles. This lets the agent estimate the complete cost of collecting and later delivering the parcels. Let C be the set of currently carried parcels, b a batch of parcels located on the same tile, and δ the estimated reward decay per movement.
For a set of parcels S delivered after D movements, we define the estimated reward for delivering a set of parcels after a given amount of movements.

$$R(S, D) = \sum_{p \in S} \max(0, r_p - \delta D) \cdot m_{parcel}(r_p)$$

Where r_p is the current estimated reward for p. The parcel multiplier is equal to one when no strategy is active.

3.1.1 Pickup Utility

A pickup action collects every parcel on the current tile, so parcels sharing the same coordinates are evaluated as a batch. A batch is eligible only when:

- its tile is reachable.
- the amount of parcels it contains is allowed by the carry stack strategy.
- the agent's partner hasn't claimed it.
- at least one reachable delivery tile from the batch's position produces a positive expected utility.

For batch b and delivery tile d, the total distance is:

$$D(b, d) = d(agent, b) + d(b, d)$$

The pickup utility is:

$$U_{pickup}(b, d) = \frac{[R(b, D) + R(C, D)] \cdot m_{stack}(|C| + |b|) \cdot m_{delivery}(d)}{\max(1, D)}$$

The reward of the carried parcels is included because they also decay while the agent reaches the batch and then the delivery tile.
For each eligible batch, the delivery tile producing the highest positive utility is chosen, and the resulting pickup desire is added to the desire set. We note that the selected delivery is not necessarily the closest one.

3.1.2 Delivery Utility

This desire can be generated if the agent is carrying at least 1 parcel and the current stack strategy allows delivery. For a delivery tile d at distance D, its utility is:

$$U_{delivery}(d) = \frac{R(C, D) \cdot m_{stack}(|C|) \cdot m_{delivery}(d)}{\max(1, D)}$$

Only delivery candidates with positive expected utility are retained. Operational delivery tiles are prioritised because they allow the agent to continue a repeatable pickup-delivery cycle. When no operational delivery is available, all reachable deliveries act as a fallback.

3.1.3 Exploration Utility

This desire is generated only when no valid pickup and delivery desires can be generated. For this utility we only consider spawners that are reachable, outside of the current sensing area and connected to an operational delivery tile.
Their utility is computed with the following formula.

$$U_{exploration}(s) = \frac{movesSinceCheck(s)}{\max(1, d(\text{agent}, s))}$$

This formula favours spawners that have not been observed recently while still paying mind to their distance. Exploration may also be generated when the agent carries parcels but no valid delivery is currently available, preventing it from remaining indefinitely idle.

3.2 Intention Loop

The intention loop continuously alternates deliberation, planning, execution and belief revision. During each iteration, the agent:

1. generates the desires that remain valid under the latest beliefs and active strategies.
2. filters desires that are temporarily obstructed or currently unplannable.
3. revises or selects the current intention.
4. plans and executes the intention's action.
5. reconciles its outcome with the beliefs, planning state and any external objective.

3.3 Intention Revision

The current intention is matched against newer desires through an identifier. If no matching valid desire remains, a new intention is selected. Otherwise, the following rules are applied:

- **External objectives**: an active objective introduced by the LLM has priority over autonomous desires and remains selected while it is valid.

- **Crate-task commitment**: a delivery using an active crate handling task is preserved until the task, intention or crate configuration changes.
- **Immediate pickup**: a valid batch on the current tile can be selected because picking it up requires no additional costs.
- **Exploration persistence**: an exploration intention is maintained while its corresponding desire remains valid.
- **Utility improvement**: an intention replaces the current intention only when its utility is strictly greater.
- **Delivery protection**: an active delivery intention can be replaced only by a pickup that has both greater utility and shorter distance than the current delivery.

If none of these replacement conditions is satisfied, the current intention is maintained.

4 Planning

Once an intention is established by the agent, the planner translates it into an executable action. For ordinary navigation, the path is reconsidered during every BDI cycle and only the next movement is returned. For routes involving crates, the planner may instead maintain a sequence of PDDL actions, which is still executed and validated one step at a time.
We decided to use two planning methods. Breadth First Search (BFS) is the default choice for navigation, while PDDL is used only when the current intention requires interacting with one or more crates.

4.1 BFS Navigation

Breadth First Search is the default navigation method, including on maps containing crates. During ordinary navigation, known crates are treated as blocked tiles, so the agent first tries to find a route around them.
BFS search is optimal for domains where the costs for all actions are equal, so it returns shortest paths in our case. The search follows the directed topology of the map: walls and non traversable tiles are excluded, directional tiles only allow movements compatible with their direction, and tiles forbidden by an active strategy are not expanded.
As explained previously, computations need the distance from the agent to several parcels, delivery tiles and spawners. A single BFS from the agent's position computes these distances. In our architecture, using A* for the same purpose would require added costs, while returning paths of the same length given our optimality guarantees.
After an intention has been selected, the planner performs a dedicated BFS towards its target and returns the first movement of the resulting path. If the target has already been reached, it returns the corresponding action, such as pickup or putdown. Only one physical action is executed before the following BDI iteration, allowing changes in beliefs or intentions to affect the next decision.

4.1.1 Dynamic Obstacles

Crates and other agents are handled differently. A known crate is a structural obstacle, since passing through it may require an explicit push plan. Another agent is instead considered as a temporary obstacle.
If the next tile is occupied, we search for a detour that avoids the obstruction while still treating

crates as blocked. If no detour exists, the agent waits rather than immediately declaring the target unreachable. When the obstruction persists, the intention is temporarily deferred. Another desire can then be selected, while the original target remains available for a later attempt.

4.2 PDDL Crate Planning

In some maps, a crate blocks the only available route and reaching the target requires moving it rather than going around it. This is the situation reserved for PDDL planning.

4.2.1 Domain Model

The domain requires only two actions. The **move** action moves the agent to an adjacent crate free tile. The **push** action requires the agent to stand behind a crate, with a valid and free destination in front of it. The end result moves the crate to the destination and the agent onto the tile previously occupied by the crate. Direction is represented through the **adjacent** and **push-line** information generated from the map. This keeps the domain limited to two general actions instead of defining separate actions for every possible direction.

4.2.2 Detecting a Route Blocked by a Crate

The planner first runs an ordinary BFS with every known crate treated as a blocked tile. If this search succeeds, the agent follows the BFS route and the solver is not used.
If no ordinary route exists, the planner searches for a possible crate corridor. When a corridor is found, the agent creates a **crate task** and uses the planner to find a solution. If none is found, a global PDDL request is used as fallback.

4.2.3 Plan Validation

PDDL planning is more expensive than ordinary BFS and the beliefs may change while a plan is being generated or executed. For this reason, the returned plan is treated as a result that must be continuously validated.

5 LLM Agent

The second part of the project requires an agent that can interpret objectives expressed in natural language, use the current game context and select the actions needed to complete a mission.
Our LLM agent receives missions through the game chat, evaluates whether they should be performed, and interacts with the environment through a catalog of tools.
The main architectural choice is that the language model never controls the environment directly. All movement, pickup and putdown actions are performed by the BDI executor. When the model requests an action, the LLM layer creates an external objective that enters the agent's BDI loop, which revises its intention, plans the next action, executes it and reconciles its outcome. Inside the LLM agent the language layer and the BDI loop share the same beliefs, planner and objective, while the teammate keeps its own independent state. LLM requested actions consequently reuse the existing pathfinding, collision handling and belief revision instead of duplicating them.

5.1 Components and Mission Lifecycle

Following the architecture introduced during the course, we separated the agent into components with distinct responsibilities:

- **LLMClient**: OpenAI's model interface.
- **LLMMemory**: builds the model context from the goal, the latest beliefs, the active strategies and recent observations.
- **LLMPlanner**: drives the execution loop, asking the model for one tool at a time or a final answer.
- **LLMReplanner**: converts a failed result into corrective feedback for the next turn.
- **LLMExecutor**: validates and executes the tools and connects them to the BDI layer.

A coordinator class combines them and manages the mission lifecycle. Only one mission runs at a time. A request arriving while the agent is busy receives an immediate reply rather than being queued or silently dropped. At the end of every mission, the coordinator cancels any objective still active and clears the mission memory.

5.2 Memory

The prompt context is rebuilt before every model call. The state section reports the agent's information similarly to what was explained during the Belief section of this report. The history keeps up to 8 entries, containing tool outcomes, formatting reminders and corrective observations produced during the current mission.
We deliberately avoid resending the complete raw conversation at every turn. A full history would grow continuously and would carry repeated or outdated state descriptions. The parcel and delivery lists are capped for the same reason, so that crowded maps do not make the context unnecessarily large.

5.3 Execution Loop and Replanning

For the reasoning process we use a **ReAct style textual execution loop**. At every turn the model must produce exactly one of two forms, and nothing else:

```
Thought: <what you are doing and why>  |  Thought: <what you concluded>
Action: <the name of one tool>         |  Final Answer: <the reply>
Action Input: <its input>              |
```

The runtime parses the output, executes the requested tool, adds its result to the mission memory as an observation and calls the model again. The loop ends when a final answer is produced, which is then returned to the mission sender through the chat.
If a response contains both an action and a final answer, the action is executed and the answer discarded. If several actions are present, only the first is executed, since the others were generated without observing their outcome.
A response matching neither form is treated as a recoverable formatting error: the runtime asks again. The loop is bounded on two sides so that an invalid mission cannot continue indefinitely. A mission may use at most ten model turns, and a failed model call is retried up to three times before the mission is stopped.
The temperature is set to 0.1, since tool names, coordinates and inputs require consistency. When a tool fails, the replanner doesn't restart from an empty context. It instead records which call failed, why, and that a different approach must be chosen. This provides semantic feedback to the model, while the replanning decision itself remains within the LLM agent.

5.4 Actions as BDI Objectives

Atomic physical missions are translated into external BDI objectives instead of being executed by the model.

Level	Tool	Purpose
1	`go_to`	Reach a specified tile through the BDI planner
1	`pick_up`	Pick up the parcels on the current tile
1	`put_down`	Put down the carried parcels on the current tile
1	`calculate`	Evaluate a restricted arithmetic expression
2	`set_strategy`	Apply a persistent behavioural or routing rule
3	`rendezvous`	Move both agents near a position and wait for both
3	`handoff_parcel`	Transfer and deliver the same parcel with two agents

Table 1: Tool catalog organised by mission level.

The first mission level covers atomic physical requests, arithmetic expressions and direct questions. An **objective** carries a unique identifier, a type and the fields that type requires. Navigation objectives carry a target, timed coordination objectives also carry an expiration time. Each request exposes a promise only when the BDI layer settles the objective. External objectives are represented as high utility desires and prioritised during intention revision, the BDI layer requires no change.

5.5 Persistent Strategy Adaptation

The second mission level introduces persistent changes to the agents' behaviour. A strategy remains active until it is replaced or explicitly cleared. The LLM agent interprets the request and applies a validated rule inside the belief state, which the BDI layer then applies deterministically in every cycle.
We defined four rule types:

- **stack**: a number of parcels to deliver together, with its reward multiplier.
- **delivery**: a multiplier assigned to one or more delivery tiles.
- **parcel value**: a multiplier for parcels above or below a given threshold.
- **avoided tile**: a tile the agent must not traverse.

A multiplier can be zero, which is how a mission states that a target yields nothing.
These rules do not replace the utility functions described in the Desire section of this report, they instead influence the expected rewards and the conditions under which desires are generated. The **avoided-tile** rule instead acts at the planning level. We made it so that both the BFS and PDDL plannings can't return a plan that goes through the given tile. In our implementation, these strategies are applied to both teammates. The validated operation is sent to the partner and applied to its own rule store.

5.6 Coordinated Missions

The two agents already cooperate during autonomous play as previously mentioned.
Level 3 missions extend this cooperation through explicit coordinated objectives, of which we

implemented two.
The **Rendezvous** mission asks both agents to reach the neighbourhood of a position within a maximum Manhattan distance and wait for each other. The tool selects two distinct free tiles inside the radius, each reachable by the agent it is assigned to. One hold objective is installed locally and the other is sent to the partner. Each agent reaches its target through its own BDI loop, and holds until the partner reaches its target. The tool completes when both agents report positions that satisfy the given radius.
The mission **Parcel handoff** requires one agent to acquire a parcel and the other to deliver that same parcel. The tool first searches for a configuration that can satisfy the mission. We give the roles of handler and receiver dynamically given the current state.
During the exchange, the handler reaches the exchange tile, puts down the selected parcel and steps onto an exit tile. The receiver waits on an adjacent tile until the parcel is on the exchange tile and the handler has left it. It then collects and delivers it. Parcel identity is preserved throughout, so the mission cannot succeed by transferring one parcel and delivering another. After leaving the exchange tile the handler stays in the hold phase until the receiver reports a terminal result. We apply a deadline derived from the complete route the agents must traverse.

5.7 Prompt Guided Mission Evaluation

Some missions explicitly award a negative reward, and others yield unoptimal utility scores given the current agent's status. Every mission is therefore evaluated by the model before selecting any tool. If a mission is declined, a **Declined** message is returned, together with a reason for its decline.

6 Challenge Benchmarks and Conclusions

To give an evaluation of the performance of our agents, we decided to measure the BDI agent's performance by trying to replicate the environment found during the first challenge. The first challenge only involves the BDI agent. The agent has to simply collect as many points as possible in 5 minutes while competing with other agents.
We ran three maps, a standard map with only the 'classic' tiles, a map with unidirectional tiles, and a map with crates.

Table 2: BDI Only Performance

Level	Score	Enemy Agents	Maximum Parcels
26c1_1	2097	8	15
26c1_4	1804	3	8
crates_one_way	1273	0	default

For the performance of the LLM agent, the lack of a standard benchmark made us choose not to report the results of a benchmark made up by us. We obviously tested both the functionality and performance of the LLM agent, both alone and while cooperating with the BDI agent, and we found the results satisfying. The LLM agent correctly understands the rules of the game, the missions given to it and it correctly performs actions in the environment. We also lack space to give more details in this report given the 10 page limit constraint.
We left instructions in our GitHub repo to install and run our project, in the case the LLM's performance is of concern. We believe that the project's requests were satisfied, and that the overall performance can be considered quite good.